

// reusable checklist

ML project setup playbook

Run through this every time you start a new model project. Four phases:
structure the code, wire up PyCharm, hand it to git, then publish it properly on GitHub.

00 What's what

01 Structure

02 PyCharm

03 Git

04 GitHub & portfolio

00

What each piece is, and how to make it

Before following the steps, here's what every file and folder in the layout actually does, and the fastest way to create each one yourself in PyCharm or the terminal.

<code>.gitignore</code>	What it is: a plain text file listing which files/folders git should never track (large data, secrets, virtual envs, IDE config). Set it up: PyCharm → right-click project root → New → File → name it <code>.gitignore</code>. Or in a terminal: <code>touch .gitignore</code>. Paste in the rules, one pattern per line.
<code>LICENSE</code>	What it is: without one, your code is "all rights reserved" by default — nobody is legally permitted to reuse it, even in a public repo. A license grants that permission explicitly. MIT is the standard, permissive choice for portfolio projects. Set it up: when creating the repo on GitHub (step 4.1), tick "Add a license" → choose MIT. Or in PyCharm: New → File → <code>LICENSE</code> and paste the MIT template, filling in your name and the year.
<code>README.md</code>	What it is: a Markdown file that GitHub renders as your repo's homepage — the first thing anyone reading your portfolio sees. Set it up: right-click root → New → File → <code>README.md</code>. Write in Markdown (<code>#</code> for headings, <code>![alt](path)</code> for images).
<code>pyproject.toml</code>	What it is: the modern config file that names your package, lists its dependencies, and tells Python how to install <code>src/</code> as an importable package. Set it up: right-click root → New → File → <code>pyproject.toml</code>. Copy the minimal template from step 1.2, then run <code>pip install -e .</code> in the terminal so it takes effect.
<code>data/raw/</code>	What it is: your original dataset, exactly as you received it. You never edit files in here by hand — treat it as read-only. Set it up: right-click root → New → Directory → <code>data/raw</code> (PyCharm creates both levels at once). Drop your original CSV/JSON files inside.
<code>data/processed/</code>	What it is: cleaned, transformed data produced by your own code from <code>data/raw</code>. If it's ever deleted, your scripts should be able to regenerate it. Set it up: New → Directory → <code>data/processed</code>. Your <code>data_loader.py</code> functions write their output here.
<code>notebooks/</code>	What it is: Jupyter notebooks for exploration only — checking distributions, trying ideas. Not where production logic lives, since notebooks are hard to version and test. Set it up: New → Directory → <code>notebooks</code>. Inside PyCharm (Professional) right-click it → New → Jupyter Notebook.
<code>src/my_project/</code>	What it is: your actual installable Python package — the production code everything else (notebooks, tests) imports from. This is the "src layout" itself. Set it up: New → Directory → <code>src/my_project</code>, then right-click that folder → Mark Directory as → Sources Root. // inside <code>src/my_project</code> <code>__init__.py</code> — an empty (or near-empty) file that tells Python "this folder is a package". New → Python File → <code>__init__</code>. <code>config.py</code> — a small module holding shared constants (file paths, random seed, test split size) so <code>train.py</code> and <code>evaluate.py</code> both read the same values instead of duplicating them. New → Python File → <code>config</code>. <code>data_loader.py</code> — functions that read <code>data/raw</code> and return a cleaned <code>DataFrame</code>. New → Python File → <code>data_loader</code>. <code>features.py</code> — feature-engineering functions, reused identically during both training and inference. New → Python File → <code>features</code>. <code>train.py</code> — the script that loads data, fits the model, and saves it to <code>models/</code>. New → Python File → <code>train</code>. <code>evaluate.py</code> — computes metrics (R^2, RMSE) and generates the plots that go in <code>outputs/figures</code>. New → Python File → <code>evaluate</code>.
<code>models/</code>	What it is: where trained model files (<code>.pkl</code>, <code>.joblib</code>) land. These are binary and derived from data + code, so they're usually gitignored rather than committed. Set it up: New → Directory → <code>models</code>. Add <code>models/*.pkl</code> to <code>.gitignore</code>.
<code>tests/</code>	What it is: unit tests that check individual functions in <code>src/</code> behave correctly, run with <code>pytest</code>. Even 2-3 tests shows real engineering discipline. Set it up: New → Directory → <code>tests</code>. Create files named <code>test_*.py</code> so <code>pytest</code> auto-discovers them, then run <code>pytest</code> from the

terminal.

outputs/figures/

What it is: the plots your `evaluate.py` generates (residual plots, confusion matrices, etc.) — small enough to commit, useful for your README.

Set it up: New → Directory → `outputs/figures`. Save PNGs here with `fig.savefig(...)` as shown earlier.

01 Structure the project (src layout)

1.1 Create the folder skeleton

Your importable code lives under `src/<package_name>` — never loose in the root. This is what separates "a folder of scripts" from an actual package.

```
my_project/
├── .gitignore
├── LICENSE
├── README.md
├── pyproject.toml
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
│   └── 01_edu.ipynb
├── src/my_project/
│   ├── __init__.py
│   ├── config.py
│   ├── data_loader.py
│   ├── features.py
│   ├── train.py
│   └── evaluate.py
├── models/
├── tests/
└── outputs/figures/
```

1.2 Add pyproject.toml

This turns `src/my_project` into an installable package so imports work identically from notebooks, scripts, and tests.

```
pyproject.toml (minimal)
[project]
name = "my_project"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = ["pandas", "scikit-learn", "joblib"]

[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"
```

1.3 Write .gitignore before you write any code

Do this before your first commit — not after. It stops raw data, model binaries, and PyCharm config from ever entering version control.

```
# .gitignore essentials
__pycache__/
.venv/
.idea/
data/raw/*
models/*.pkl
*.ipynb_checkpoints/
```

02 Wire it up in PyCharm

2.1 Create a fresh virtual environment

File → Settings → Project → Python Interpreter → Add Interpreter → Virtualenv Environment → New. One venv per project, always.

2.2 Install your project in editable mode

```
PS C:\my_project> pip install -e .
```

Why: this reads `pyproject.toml` and lets you write from `my_project.train import main` anywhere in the project, including notebooks.

2.3 Mark src/ as the Sources Root

Right-click the src folder in the Project pane → Mark Directory as → Sources Root. PyCharm usually detects this automatically once pyproject.toml exists, but check it manually if autocomplete misbehaves.

03 Hand it to git

3.1 Enable version control

VCS → Enable Version Control Integration → Git. File names turn red in the Project pane — that means untracked, not broken.

3.2 Stage and commit

Use small, descriptive commits rather than one giant dump at the end — it reads far better in your history.

```
PS C:\my_project> git add .
PS C:\my_project> git commit -m "Add src layout and data loader"
```

In PyCharm: use the Commit tool window (**Cmd/Ctrl+K**) instead of the terminal if you prefer checkboxes over typing.

04 Publish to GitHub & portfolio it

4.1 Create an empty repo on GitHub

github.com → + → New repository → same name as your local folder. Leave it empty — no README, no .gitignore — to avoid a merge conflict on first push.

License: tick "Add a license" while creating the repo and choose MIT — it's the standard permissive choice for portfolio projects. If you skip it here, add a LICENSE file locally afterward instead.

4.2 Connect and push

```
PS C:\my_project> git remote add origin https://github.com/you/my_project.git
PS C:\my_project> git push -u origin main
```

In PyCharm: VCS → Git → Push (**Cmd/Ctrl+Shift+K**) after adding the remote under VCS → Git → Remotes.

4.3 Write the README that recruiters actually read

What the project does, the dataset, your approach, key results (R^2 , RMSE, etc.), a plot, and exact run instructions.

4.4 Polish for the portfolio

Pin the repo on your GitHub profile, add topics like machine-learning and scikit-learn, and embed a results image directly in the README.

// quick checklist — tick through before every push

- src/ layout with pyproject.toml
- .gitignore written before first commit
- venv created and pip install -e . run
- src/ marked as Sources Root in PyCharm
- LICENSE added (MIT for most portfolio projects)
- Small, descriptive commits
- README with results and run instructions
- Repo pinned + topics added on GitHub
- No data/model binaries committed